Research Project
(Comprehensive Design and Analysis Project)

IT4010



KADUNA.LK: A UNIFIED SMART PLATFORM FOR ROADSIDE ASSISTANCE AND AUTOMOTIVE SERVICES IN SRI LANKA

Deployment Report

Group Details

Group No: R26-SE-026

Student ID	Student Name
IT22635266	Janukshan S
IT22633422	Asath M M
IT22639776	Herath D M S T
IT22001252	De Silva R K D H

Table of Contents

1. DEPLOYMENT OVERVIEW 1
1.1 Overview 1
1.2 Purpose of the Deployment 1
1.3 Scope of the Deployment 2
1.4 Key Metrics Monitored 2
1.5 Deployment Success Metrics 3
1.6 Success Rate and Error Monitoring 3
1.7 Key Outcomes of the Deployment 4
2. DEPLOYMENT PROCESS 5
2.1 Deployment Environment Preparation 5
2.2 Platform Initialisation 6
2.3 Deployment of System Components 6
2.4 Secure Access Configuration 9
2.5 Execution of Testing Scenarios 10
2.5.1 Scenario 1 – End-to-End Emergency Spine 10
2.5.2 Scenario 2 – Traffic-Impact Scoring and Road-Class Resolution 11
2.5.3 Scenario 3 – Predictive Maintenance Inference 11
2.5.4 Scenario 4 – Web Portals and Role-Based Access 11
2.5.5 Scenario 5 – Accident Claim Capture Through a Web Link 12
2.5.6 Scenario 6 – Release-Build Verification 12
2.6 Backup and Recovery Strategy 12
2.7 Deployment Details 12
2.8 Deployment Checklist 13
2.9 Deployment Verification and Completion 14
3. ISSUES, RESOLUTION, AND MONITORING 15
3.1 Issues Encountered During Deployment 15
3.2 Resolutions Applied 16
3.3 Impact Analysis 17
3.4 Monitoring and System Observability 17
3.5 Monitoring Results 18
3.6 User Feedback 18
4. POST-DEPLOYMENT VALIDATION, DOCUMENTATION, AND LESSONS LEARNED 20
4.1 Post-Deployment Validation 20
4.2 Validation Activities 20

4.3 Supporting Operational Artefacts 21
4.4 Key Lessons Learned 21
4.5 Known Limitations and Next Steps 22
4.6 Conclusion of Post-Deployment Assessment 22

1. DEPLOYMENT OVERVIEW

1.1 Overview

This report describes how the Kaduna.lk platform was deployed and verified. Kaduna.lk is a roadside-assistance and automotive-services platform for Sri Lanka. It brings four research components together: dispatch optimization, geo-intelligence and traffic impact analysis, predictive maintenance, and accident-claim capture. Drivers, service providers, operations staff and insurers use it through an Android application and the web application at kaduna.lk.

The platform runs in production on the public internet, not on a laboratory cluster. The three backend services and the web application run on a Contabo virtual private server (VPS) managed by Dokploy, a self-hosted platform-as-a-service that runs on Docker Swarm, with Traefik as the edge proxy. Supabase provides authentication, the PostgreSQL database and the serverless Edge Functions used by the claims component. The Android application is distributed as a signed release APK.

Every application is built from the team's Git repository (github.com/JanukshanS/R26-SE-026, branch dev), which makes the deployment repeatable and lets every running build be traced to a commit.

1.2 Purpose of the Deployment

The deployment serves three purposes.

- **To validate the integrated platform under real conditions.** The research components were developed separately. Deployment shows that they work together across real networks, real TLS and real authentication. The key path is what the team calls the *emergency spine*: a driver reports an incident in the mobile app, the dispatch service ranks service providers, and dispatch asks the geo-intelligence service for a traffic-impact score that feeds into the provider ranking.
- **To give stakeholders a working system.** The live web application was used in the stakeholder sessions held during evaluation, including a live demonstration to a senior Department of Motor Traffic (DMT) official on 5 October 2026.
- **To show that the system can be operated.** Health checks, rolling updates, automatic rollback and a written release procedure show that the platform can be maintained beyond the research period, not only demonstrated once.

1.3 Scope of the Deployment

The deployment covers:

- Provisioning and hardening the VPS, and installing Dokploy and Traefik.
- Deploying three backend services (dispatch, geo-intelligence, predictive maintenance) and the web application, each as its own container image.
- Configuring the kaduna.lk domain, sub-domains and automatically renewed TLS certificates.
- Configuring Supabase Auth (email/password and Google sign-in), PostgreSQL with row-level security, and three Edge Functions for the claims flow.
- Building, signing and installing the Android release APK on a physical device.
- Continuous integration on GitHub Actions for the components on the emergency spine.

The deployment does **not** cover Google Play Store distribution, multi-node high availability or formal load testing. These are listed as limitations in Section 4.5.

1.4 Key Metrics Monitored

The following metrics were monitored during and after deployment to judge whether the platform was available, correct and secure:

- **Service health.** Each backend service exposes a health endpoint that reports its status and its dependencies: database connectivity and uptime for dispatch; the version, Git commit, model weights and loaded datasets for geo-intelligence; and the loaded models for predictive maintenance.
- **HTTP status codes and response times** of every public endpoint, measured from Sri Lanka.
- **Authentication enforcement.** Protected routes must return 401 Unauthorized without a valid Supabase access token.
- **Scoring latency** of the geo-intelligence service, in process and end to end.
- **Container health-check state** and the status of each rolling update in Dokploy (running, done or error).
- **Continuous-integration results** for every change to the components on the spine.

- **End-to-end correctness of the spine.** The dispatch decision must report its traffic-impact source as geo-intelligence, not the neutral fallback.

1.5 Deployment Success Metrics

In addition to the runtime metrics, the operational indicators in Table 1.1 were used to judge whether the deployment succeeded.

Table 1.1: Deployment success metrics

Metric	Description	Result
Deployment success rate	Share of applications deployed and passing their health checks	4 of 4 VPS applications and 3 of 3 Edge Functions live on 14 October 2026
Error rate	Container crashes, failed health checks or failed rolling updates in steady state	None observed at the final verification; earlier errors are listed in Chapter 3
Authentication enforcement	Protected routes reject unauthenticated calls	POST /v1/score without a token returns 401
Spine integrity	Dispatch uses the live geo-intelligence score, not the fallback	Verified in production; most recently re-verified on 9 October 2026
Release-build correctness	The mobile bundle points at production, not localhost	Verified by extracting the embedded endpoints from the APK
Automated test gate	CI passes on dev	Latest dev run successful (9 October 2026)

1.6 Success Rate and Error Monitoring

The success rate was judged by whether each application started, passed its health check and served correct responses. During the deployment:

- All four VPS applications (kaduna-web, kaduna-dispatch, kaduna-geo and kaduna-predict) were built by Dokploy from the repository and passed their health checks.
- The three claims Edge Functions (sign-photo-upload, complete-capture and verify-claimant) were deployed to Supabase and report status ACTIVE, with JWT verification enabled.
- TLS certificates were issued automatically for kaduna.lk, www.kaduna.lk and every service sub-domain.
- The release APK was installed on a Google Pixel 7 and completed the emergency spine against production.

Several errors occurred during the earlier phases. Among them were an IPv6-only database endpoint, a web container that its own health check kept restarting, and an empty providers

table after a migration. All were diagnosed and fixed; Chapter 3 records each one. None of them remains in the final deployment.

1.7 Key Outcomes of the Deployment

- The full platform is live at a real domain with TLS, serving four research components through one web application and one mobile application.
- The emergency spine works end to end in production: the dispatch service fetches the geo-intelligence score server-side and forwards the caller's own token.
- Every service checks the caller's Supabase token against the project's public keys. No service holds a signing secret.
- Releases are repeatable. Images are built from Git, rolling updates start the new task first, and an update that fails its health check rolls back automatically.
- Stakeholder feedback from the live system led to a concrete fix: road class is now resolved from the incident's GPS position (Section 3.1, Issue 7).

2. DEPLOYMENT PROCESS

The deployment was carried out in phases: environment preparation, platform initialisation, component deployment, secure-access configuration and scenario-based validation. This chapter describes the tools, configuration and steps actually used.

2.1 Deployment Environment Preparation

The production environment is a Contabo cloud VPS. Table 2.1 lists its specification.

Table 2.1: Production environment

Item	Value
Host	Contabo VPS, European region (Germany), single node
Operating system	Ubuntu 24.04.4 LTS
Resources	6 vCPU (AMD EPYC, 2.0 GHz), 12 GB RAM, 200 GB SSD
Platform	Dokploy v0.30.3 (self-hosted PaaS on Docker Swarm)
Edge proxy	Traefik v3.6.7, with TLS termination and routing by hostname
TLS	Let's Encrypt certificates, issued and renewed automatically; HTTP/2 enabled
Domain	kaduna.lk for the web application (www redirects to the apex); backend services at <service>.vps.kaduna.lk
Identity provider	Supabase Auth (GoTrue), project kaduna, region ap-northeast-2
Database	Supabase PostgreSQL, reached through the Supavisor IPv4 connection pooler
Server access	SSH with key authentication only; password login disabled

The server size was chosen so that all services and their image builds fit on one host. Predictive maintenance needs the most memory, because it loads its scikit-learn models into memory.

The deployment went through two stages. The first version used docker compose with a Caddy reverse proxy and temporary sslip.io hostnames, while the kaduna.lk DNS records were still being set up. On 22 August 2026 the platform moved to Dokploy, which added per-application deployment history, automatic TLS for the real domain and Git-triggered builds. The old compose directory was renamed so that the old deployment script would fail loudly if run by mistake, and a backup of its environment was kept on the server.

2.2 Platform Initialisation

Dokploy. Dokploy was installed on the VPS, and one project was created to hold the platform. Each application was registered separately, linked to the GitHub repository and branch dev, and pointed at its component's Dockerfile. Domains were attached to each application, and Traefik obtained certificates on the first request to each one.

Supabase. The Supabase project provides:

- Authentication with email/password and Google OAuth (PKCE flow for mobile; redirect and One Tap for web).
- PostgreSQL tables for profiles, vehicles, incidents and related records, protected by row-level security. The migration web_roles_and_role_lockdown added a role check constraint, removed the role column from what clients can update, and added an is_ops() helper for operations-only policies.
- Separate schemas for the dispatch service (managed by Prisma) and for predictive maintenance.
- Edge Functions and storage for the claims component.

Because all persistent state lives in Supabase and none on the VPS, a rebuilt server is restored simply by redeploying the images.

2.3 Deployment of System Components

Each component is deployed as its own container image, built by Dokploy from the Dockerfile in the repository. The components are described below. Table 2.2 gives an inventory.

Table 2.2: Application inventory

Application	Public URL	Owner	Runtime
kaduna-web	kaduna.lk	Shared	Static Next.js export served by nginx
kaduna-dispatch	dispatch.vps.kaduna.lk	Janukshan S	Node.js / Express / Prisma
kaduna-geo	geo.vps.kaduna.lk	Asath M M	Python / FastAPI
kaduna-predict	predict.vps.kaduna.lk	Herath D M S T	Python / FastAPI
Claims Edge Functions	Supabase project kaduna	De Silva R K D H	Deno (Supabase Edge Functions)
Mobile application	Release APK (sideloaded)	Shared	Expo / React Native, Android

Dispatch Optimization and Smart Recovery service

- Image node:18-alpine: npm ci, prisma generate and a TypeScript build. Express listens on port 3001.
- The start command runs prisma migrate deploy before the server starts, so schema migrations are applied on every deployment.
- Configuration: the database URL (schema dispatch), SUPABASE_URL, GEO_INTELLIGENCE_URL, the Google Maps key used for routing, and the log level.
- Health check: GET /health every 10 s, with a 3 s timeout, a 20 s start period and 5 retries. The endpoint reports database connectivity and uptime.
- Fifteen service providers were registered in production through the providers API, so that dispatch has real candidates to rank.

Geo-Intelligence and Traffic Impact Analysis service

- Image python:3.11-slim, served by uvicorn on port 5001. Version 0.2.0 is deployed.
- The service is stateless. The scoring model, the Colombo OpenStreetMap road index (34,265 road ways), the hotspot clusters, the points-of-interest data for the sensitivity overlay and the Sri Lanka holiday calendar (2026 and 2027) are built into the image. It needs no database and no volume.
- Configuration: SUPABASE_URL only. No secrets are deployed with this component.
- Health check: GET /v1/health every 10 s. The endpoint reports the version, the Git commit, the deployed weights and the loading state of each dataset, so it is easy to confirm which build is running.
- The deployed weights are the original expert weights (capacity loss 0.25, traffic volume 0.25, temporal 0.20, location 0.15, incident severity 0.15). The weights fitted to SUMO simulation in the research study are a sensitivity result and are not deployed.

Predictive Maintenance for Service Management service

- Image python:3.11-slim, served by uvicorn on port 5000.
- Eight trained models (random forest and gradient boosting for each of engine, brake, tyre and battery) are loaded at start-up. The health endpoint reports the best model for each part.
- Persistence is in its own Supabase PostgreSQL schema, selected by an explicit SET search_path on each connection (see Section 3.1, Issue 6).
- Configuration: the database URL, SUPABASE_URL and the Supabase publishable key, which is used to check the operations role on catalogue routes.

Intelligent 3D Accident Claim and Privacy-Enforced Ecosystem

- The claim-capture flow runs inside the web application. A driver receives a short claim link (kaduna.lk/c/…), delivered by SMS through Notify.lk, and completes a guided photo capture in the browser, without installing the app. Insurers review claims under /insurer.
- Three Supabase Edge Functions handle the server side. verify-claimant checks the claimant, sign-photo-upload issues signed upload URLs so that photos go straight to storage, and complete-capture finalises a capture. All three have JWT verification enabled.
- Uploads are queued in the browser's IndexedDB, so a capture can resume after a dropped connection without losing photos.
- The earlier stand-alone claims-privacy FastAPI service has been retired in favour of this design and has no remaining callers.

Web application (kaduna.lk)

- A two-stage build: node:22-alpine builds a static Next.js export, and nginx:1.27-alpine serves it. The build context is the repository root, because the TypeScript configuration refers to the shared contracts/ directory.
- Routes: the public landing page /, driver portal /app, provider console /provider, incident report /report, operations dashboard /dashboard, administration /admin, insurer view /insurer, claim flow /claim and /c, and the architecture diagrams at /diagrams/.

- NEXT_PUBLIC_* values are inlined at build time, so changing one needs a rebuild, not a restart. All of them are publishable values by design.
- This is the only application that redeploys automatically on a push to dev.

Mobile application

- Package com.kaduna.app, version 1.1.0, minimum SDK 24. It is built locally with Gradle through apps/mobile/build-release.sh.
- The build script first moves .env.local aside, because Expo gives that file precedence and would otherwise put a localhost endpoint into the release build (Section 3.1, Issue 4).
- The release bundle embeds the dispatch and predictive-maintenance URLs and the Supabase project URL. The geo-intelligence URL is intentionally absent, because the app never calls geo-intelligence directly: dispatch calls it on the server.

2.4 Secure Access Configuration

The platform is publicly reachable, so access control works at several layers instead of relying on keeping services private.

Server and platform access. The VPS accepts SSH with key authentication only. Deployments are made through the Dokploy console, which is served over HTTPS, or through its API from the server itself. Only Traefik publishes ports 80 and 443. The service containers are not exposed directly.

Transport security. Traefik terminates TLS for every hostname using Let's Encrypt certificates that renew automatically. Plain HTTP is redirected to HTTPS.

Service authentication. Every data route requires Authorization: Bearer <Supabase access token>. Each service downloads the project's public keys (JWKS) from ${SUPABASE_URL}/auth/v1/.well-known/jwks.json and checks the token's signature, issuer, audience and expiry. Health endpoints stay open on purpose, so that container probes and post-deployment checks work without credentials. If SUPABASE_URL is missing, a service returns 503 rather than serving unauthenticated requests. Table 2.3 shows how each caller obtains a token.

Table 2.3: How callers authenticate

Caller	How it obtains a token
Mobile application	Its Supabase session, attached to each request
Web application	Sign-in gate; operations-only routes also check profiles.role
Dispatch calling Geo-Intelligence	Forwards the caller's Authorization header unchanged
Claims Edge Functions	Supabase verifies the JWT before the function runs

Data authorisation. Row-level security in PostgreSQL limits each user to their own profiles, vehicles and incidents. Operations users can read everything through the is_ops() policy. Clients cannot change their own role.

Pending hardening. A Supabase custom access-token hook, which adds the platform role to the token, has been merged into the repository (PR #67). Until it is enabled in the Supabase console, geo-intelligence runs with role enforcement switched off (GEO_ENFORCE_ROLES=false) and requires only a valid token.

2.5 Execution of Testing Scenarios

After deployment, the platform was tested with six scenarios, each aimed at a different part of the architecture.

2.5.1 Scenario 1 – End-to-End Emergency Spine

This scenario tests the main integration path across components.

Test conditions

- Release APK on a physical Google Pixel 7, signed in with a test account, against production services.
- The driver reports an incident and completes triage. The app calls POST /api/v1/dispatch/optimize and does not send a traffic-impact score of its own.

Expected behaviour

- Dispatch calls geo-intelligence POST /v1/score on the server, forwarding the driver's token, with a 2 s timeout and a neutral fallback score of 5.
- The decision reports source: geo-intelligence and a positive traffic-externality cost in the provider ranking.

Result. A provider was assigned, and the live geo-intelligence score was shown in the app. After geo-intelligence 0.2.0 was deployed, the spine was re-verified in production on 9

October 2026: dispatch logged a score of 9.2 from geo-intelligence rather than the fallback. This confirms the contract, the token forwarding and the fallback path. It does not show that production decisions improved, because the bounded traffic-impact term cannot reorder otherwise identical providers.

2.5.2 Scenario 2 – Traffic-Impact Scoring and Road-Class Resolution

This scenario tests the geo-intelligence service directly with authenticated requests.

- **Road class from GPS.** The same engine-failure incident (one lane blocked, Monday 08:00) scores 7.8 on a trunk road in Colombo Fort and 5.5 on a residential street in Nugegoda. An offshore point correctly falls back to the default road class.
- **Sensitivity overlay.** On 10 April 2026, a holiday, an incident near the National Eye Hospital gets the capped sensitive-location multiplier (×1.225).
- **Access control.** The same request without a token returns 401.
- **Latency.** Across 1,000 warmed worst-case calls, the in-process 95th-percentile scoring time was 9.7 ms. A single end-to-end call from Sri Lanka to the European server took 1.45 s, which misses the 500 ms target set for the component (Section 4.5).

2.5.3 Scenario 3 – Predictive Maintenance Inference

- The health endpoint confirms that all eight models are loaded, with the best model per part reported as random forest for engine, brake and battery and gradient boosting for tyre.
- Authenticated prediction requests return a component-level maintenance prediction.
- The catalogue routes reject users who do not have the operations role.

2.5.4 Scenario 4 – Web Portals and Role-Based Access

- kaduna.lk and kaduna.lk/dashboard return 200, and www.kaduna.lk returns 301 to the apex domain.
- A signed-in driver can reach /app and /report. A provider receives dispatch contracts in /provider, which polls every 5 s and marks the provider offline on sign-out.
- /dashboard and /admin are refused to users without the operations role. Administrators change roles through the admin_set_role database function, not by direct table writes.

2.5.5 Scenario 5 – Accident Claim Capture Through a Web Link

- An insurer or operator generates a claim link, which reaches the driver by SMS through Notify.lk.
- The driver opens kaduna.lk/c/…, is verified by verify-claimant, and captures the guided photo set. Photos are uploaded through signed URLs from sign-photo-upload, and complete-capture finalises the claim.
- If the connection drops in the middle of a capture, it resumes from the IndexedDB upload queue without losing photos.

2.5.6 Scenario 6 – Release-Build Verification

- The JavaScript bundle was extracted from the release APK and its embedded endpoints were checked: they point at the production hostnames, not at localhost.
- The APK was installed on the Pixel 7 and used for Scenario 1.

2.6 Backup and Recovery Strategy

- **Code and configuration in Git.** Every image is built from a commit on dev, and Dokploy records the commit for every deployment, so any earlier build can be redeployed.
- **State outside the server.** All persistent data is in Supabase. The VPS holds no data that cannot be rebuilt.
- **Safe rolling updates.** Swarm services update start-first: the new task must pass its health check before the old one stops, and a failed update rolls back automatically.
- **Rollback.** Redeploy the last known-good commit of the affected application. Only dispatch and predictive maintenance have database migrations, and both are applied additively, so rolling back application code does not need a schema rollback.
- **Configuration backup.** The environment of the earlier compose deployment was backed up on the server before the move to Dokploy. The current configuration lives in each Dokploy application, and .env.example in the repository lists every key and why it exists.

2.7 Deployment Details

- Deployment environment: Contabo VPS (6 vCPU, 12 GB RAM, 200 GB SSD), Ubuntu 24.04.4 LTS

- Deployment tools: Dokploy, Docker Swarm, Traefik, GitHub, GitHub Actions, Supabase, Gradle
- Topology: single-node Swarm hosting four applications, with Supabase as the managed back end
- Source: github.com/JanukshanS/R26-SE-026, branch dev
- Team: Group R26-SE-026
- Execution mode: phased deployment, from compose with Caddy (initial) to Dokploy (from 22 August 2026), followed by repeated verification rounds up to 14 October 2026

2.8 Deployment Checklist

Pre-deployment

- Server provisioned, SSH key access configured, password login disabled
- DNS records for kaduna.lk and the service sub-domains created
- Supabase project, authentication providers, row-level security policies and schemas prepared
- Environment variables prepared for each application from .env.example
- Test suites passing locally and in CI

During deployment

- Dokploy installed and each application linked to the repository
- Images built from component Dockerfiles; health checks defined
- Domains attached; TLS certificates issued by Traefik
- Edge Functions deployed to Supabase
- Production providers registered for dispatch
- Release APK built with production endpoints and installed on a test device

Post-deployment

- All health endpoints return 200 with the expected version and dependencies
- Unauthenticated calls to protected routes return 401

- Authenticated scoring returns a resolved road class
- End-to-end spine verified with source: geo-intelligence
- Web portals and role-gated routes verified
- Architecture diagrams published at kaduna.lk/diagrams/

2.9 Deployment Verification and Completion

A final verification was carried out on 14 October 2026 at 13:02 UTC from Sri Lanka. Table 2.4 shows the results.

Table 2.4: Final live verification (14 October 2026)

Check	Result
GET geo.vps.kaduna.lk/v1/health	200 in 0.90 s; version 0.2.0, commit c261b6c, original weights, road index, POI and holiday datasets loaded
GET dispatch.vps.kaduna.lk/health	200 in 1.13 s; healthy, database connected, uptime 4.4 days
GET predict.vps.kaduna.lk/health	200 in 1.02 s; 8 models loaded
GET kaduna.lk · www.kaduna.lk · /dashboard	200 · 301 to apex · 200
POST geo.vps.kaduna.lk/v1/score with no token	401, authentication enforced
Supabase Edge Functions	sign-photo-upload, complete-capture, verify-claimant all ACTIVE, JWT verification on
CI on dev	Latest Geo Spine Quality run successful (9 October 2026)

Together with the scenarios in Section 2.5, these checks confirm that the platform is deployed, reachable over TLS, secured by token, and integrated end to end.

3. ISSUES, RESOLUTION, AND MONITORING

3.1 Issues Encountered During Deployment

The team met the following technical and operational problems during deployment. Each one is recorded because it caused a real failure.

Issue 1: The database could not be reached from the containers. Supabase's direct database host (db.<ref>.supabase.co) resolves only to IPv6, and the container network had no IPv6 route. Dispatch failed with "network is unreachable", even though the same lookup worked from the host.

Issue 2: The web container was killed by its own health check. The Alpine health probe resolved localhost to the IPv6 loopback ::1, but nginx was listening only on IPv4. Every probe failed, and Swarm kept replacing a container that was actually working.

Issue 3: Maps were blank on the live site. The keyless CARTO basemap tiles were rate-limited and blocked in production, so the operations dashboard and the research dashboard showed empty maps.

Issue 4: A release APK pointed at localhost. Expo gives .env.local precedence over .env. A release built on a developer machine therefore embedded a localhost API endpoint and could not reach production.

Issue 5: Production dispatch returned "No available providers found". After the move to the new database, the providers table was empty. The emergency spine could not be shown end to end in production, although every service was healthy.

Issue 6: Predictive maintenance used the wrong schema. Prisma honours a ?schema= parameter in the database URL, but SQLAlchemy ignores it. The same style of URL therefore pointed predictive maintenance at the wrong schema.

Issue 7: Every incident was scored as if it were on a primary road. During the interview with a retired Deputy Inspector General of Police (ex-DIG), who stressed how much road importance matters, the team checked the live inputs. Every incident was reaching the model as a two-lane primary road, because callers did not supply a road class and a constant default was used.

Issue 8: Backend redeployment was manual. Only the web application redeploys automatically. After a merge, the backend services kept running the old build until someone triggered them, which could leave production behind dev.

3.2 Resolutions Applied

Resolution for Issue 1. The database connections were switched to the Supavisor IPv4 connection pooler (aws-1-ap-northeast-2.pooler.supabase.com, user postgres.<project-ref>), which needs no IPv6 inside the container network.

Resolution for Issue 2. nginx was configured to listen on [::]:80 as well as on IPv4, so the probe succeeds whichever loopback it resolves.

Resolution for Issue 3. The basemap was moved to OpenStreetMap tiles on both dashboards, with a CSS filter for the dark theme. Maps loaded again on the live site.

Resolution for Issue 4. apps/mobile/build-release.sh was written to move .env.local aside before every release build. The endpoints embedded in the bundle are now extracted and checked as part of release verification (Scenario 6).

Resolution for Issue 5. Fifteen providers were registered through POST /api/v1/providers. This added data without deleting any. The seed script was deliberately not used, because it deletes incidents, triage records and decisions before seeding. After the fix, the spine was verified end to end in production.

Resolution for Issue 6. Predictive maintenance now runs an explicit SET search_path on each connection, and the configuration documentation states that database URLs cannot be shared between services.

Resolution for Issue 7. Road resolution was moved into the geo-intelligence service. Road class and lane count are now looked up from the incident's GPS position against 34,265 bundled OpenStreetMap road ways. The fix was deployed and verified live: a Colombo Fort incident resolves to a trunk road matched 1.8 m from the coordinates.

Resolution for Issue 8. This was kept as a deliberate control rather than automated: each backend owner decides when a model or algorithm change reaches production. The release procedure now ends with a required post-deployment check, and the geo-intelligence health endpoint reports the running commit, so a stale deployment is easy to spot.

3.3 Impact Analysis

Most of the issues had a moderate impact in the early phase. Issues 1 and 2 stopped services from starting at all and blocked the first move to the new platform. Issue 5 hid the main integration result even though every individual service was healthy. This is why the post-deployment check now includes an end-to-end test, not only health checks.

Issue 7 mattered most for the research. It meant the location factor was effectively constant for every live incident. It was found only because the system was deployed and shown to a domain expert, and fixing it changed how the scores separate incidents on major roads from those on residential streets.

None of the issues required a change to the overall architecture. Once they were resolved, the platform ran stably, and the final verification (Table 2.4) found no outstanding errors.

3.4 Monitoring and System Observability

Metrics monitored

- Service status, database connectivity, uptime and loaded models, from the health endpoints
- The running version, Git commit, weights and dataset state of geo-intelligence
- HTTP status and response time of each public endpoint
- Container health-check state and rolling-update status in Dokploy
- The source of each traffic-impact score in dispatch (geo-intelligence or fallback), recorded in the scoring log with the time of the incident

Monitoring tools and techniques

- **Dokploy console**: per-application deployment history with the commit, build logs and container logs
- **Docker Swarm health checks**: automatic restart, and rollback of updates that fail
- **GitHub Actions (Geo Spine Quality workflow)**: runs on every change to geo-intelligence, dispatch, the contracts or the web application. It runs the geo-intelligence test suite, the dispatch test suite against a PostgreSQL service container, an OpenAPI contract lint, and a check that the hotspot data in the service matches the copy in the web application

- **scripts/spine-smoke.sh**: an end-to-end check that source is geo-intelligence and the traffic externality is positive
- **Operations dashboard**: scores, factors, map layers, the scoring log and the model-accuracy view
- **Supabase dashboard**: authentication events, database state and Edge Function status

The platform has no centralised metrics stack, such as Prometheus and Grafana, and no automated alerting. Monitoring relies on health checks, logs and scheduled manual verification. Section 4.5 lists this as a limitation.

3.5 Monitoring Results

Once the issues above were resolved, monitoring showed stable operation. All health endpoints returned 200 with the expected dependencies. The dispatch service reported 4.4 days of continuous uptime since its last deployment at the final check. The latest CI run on dev passed. The two failed CI runs on record (31 August 2026) were on a feature branch and were fixed before that branch was merged. Unauthenticated access to protected routes was refused consistently.

End-to-end latency is dominated by the distance between Sri Lanka and the European server: about 0.9 to 1.3 s for a health request, compared with under 10 ms of in-process scoring time. Hosting closer to users is therefore the most effective latency improvement (Section 4.5).

3.6 User Feedback

Feedback was collected on the live system from domain stakeholders.

- **Retired Deputy Inspector General of Police (27 August 2026, semi-structured interview).** He confirmed that the formal incident pathway covers expressways only and that “there is no similar system to help locals” on ordinary roads. His emphasis on road importance led directly to the road-class fix (Issue 7). He also described weekday and direction effects (heavy traffic into Colombo on Mondays and out on Fridays) that the current model does not represent.
- **Senior DMT official (5 October 2026, live demonstration and structured form).** The System Usability Scale score was 80.0, rated “good”. Factor relevance averaged 4.4 out of 5, and the usability and trust items both scored 5. The official gave clearance-time estimates that differed from the model defaults by a mean of 12.1 minutes, and stated that the DMT keeps no timed, location-coded incident records.

With a single respondent after a guided demonstration, these results are indicative only.

Suggested improvements include modelling weekday and direction effects, giving road class more weight on major roads, and using clearance times from local experts.

4. POST-DEPLOYMENT VALIDATION, DOCUMENTATION, AND LESSONS LEARNED

4.1 Post-Deployment Validation

After deployment, the platform was validated in its production environment. Administrative work was done over SSH to the VPS and through the Dokploy console. Dokploy showed the state of each application, its deployment history and its container logs, while SSH allowed direct inspection of the Swarm services and the network. The Supabase dashboard was used to check authentication, row-level security and Edge Function status.

The deployed platform consists of the dispatch, geo-intelligence and predictive-maintenance services, the web application and its claim flow, the claims Edge Functions and the Android application. The validation confirmed that each service is reachable over TLS at its public hostname, reports healthy dependencies, rejects unauthenticated calls and takes part correctly in the end-to-end flow.

4.2 Validation Activities

Validation was carried out at several levels, as Table 4.1 shows.

Table 4.1: Validation activities

Validation level	Activities performed
Infrastructure	VPS resources and Swarm services inspected; every application running with its health check passing; TLS certificates valid for every hostname; www redirect confirmed
Application	Health endpoints reviewed for version, commit, weights, models and database connectivity; container logs inspected after each deployment
Security	Unauthenticated calls refused with 401; role-gated web routes refused to non-operations users; Edge Functions verify JWTs; no signing secrets in any service
Integration	End-to-end spine verified in production with source: geo-intelligence; dispatch forwards the caller's token to geo-intelligence
User interface	Web portals verified for each role; operations dashboard displays live scores, map layers and the scoring log; mobile release APK verified on a Pixel 7
Automated	CI suite (geo-intelligence tests, dispatch tests, contract lint, data parity) passing on dev; geo-intelligence 0.2.0 passes 146 tests

These checks show that the platform works as an integrated, secured system, not only as a set of services that started.

4.3 Supporting Operational Artefacts

The repository contains the artefacts needed to reproduce and run the deployment:

- A Dockerfile for each component, with its health check
- .env.example for each application, listing every configuration key and why it exists
- OpenAPI contracts for each component in contracts/, checked by CI
- A C4 architecture model (docs/architecture/workspace.dsl) with context, container, dynamic and **deployment** views of the topology described in this report, published at kaduna.lk/diagrams/
- scripts/spine-smoke.sh for the end-to-end check, and apps/mobile/build-release.sh for safe release builds
- The CI workflow .github/workflows/geo-spine-quality.yml
- Per-component READMEs with run and test commands, and docs/flows.md describing the request flows

These make deployment repeatable and troubleshooting easier, and they give future developers a documented way to operate the platform.

4.4 Key Lessons Learned

- **Deploying to production finds defects that local testing does not.** The IPv6-only database host, the IPv6 health probe and the blocked map tiles all worked on developer machines and failed only in production.
- **Health checks alone do not prove integration.** Every service was healthy while the spine returned “No available providers found”. A post-deployment check must include one real end-to-end request.
- **Build-time configuration needs its own checks.** Values inlined at build time (NEXT_PUBLIC_* and EXPO_PUBLIC_*) cannot be fixed by a restart, so the release bundle has to be checked directly.
- **Keeping state outside the server makes recovery simple.** With all data in Supabase and models built into the images, a rebuilt server is restored by redeploying.
- **Showing the live system to experts improves the research.** The road-class defect was found only because a domain expert used the system in its deployed form.

- **Seed scripts can destroy data.** Registering providers through the API instead of the seed script avoided wiping production incidents.

4.5 Known Limitations and Next Steps

- **Single node.** All services share one VPS. A failure of the host affects the whole platform.
- **Server region.** The server is in Europe. End-to-end calls from Sri Lanka take over 1 s, which misses the 500 ms target. Hosting in a nearer region (for example Singapore) is the main improvement.
- **Mobile signing.** The release APK is signed with the debug keystore. This is enough for sideloaded evaluation but not for Play Store distribution.
- **Role enforcement.** Geo-intelligence will enforce roles once the Supabase access-token hook is enabled and GEO_ENFORCE_ROLES is set to true.
- **Observability.** There is no centralised metrics stack or automated alerting yet.
- **Load.** The platform has not been load-tested at realistic concurrency.

4.6 Conclusion of Post-Deployment Assessment

The post-deployment assessment confirms that Kaduna.lk is deployed and running in a public production environment. The three backend services and the web application run on a Dokploy-managed VPS behind Traefik with automatic TLS, and authentication, data and the claims Edge Functions run on Supabase. Every service enforces token authentication, the emergency spine works end to end in production, and the final verification on 14 October 2026 found every component healthy. The problems met along the way were diagnosed, fixed and documented. One of them, the road-class defect, directly improved the research model. The deployment is therefore functional, secure and reproducible enough to support the project’s objectives, and the limitations above define the work needed to take it towards production scale.